

Comprehensive Technical Architecture and Systems Engineering Report: Smart Trade Agentic AI System

1. Architectural Paradigm and System Context

1.1 The Evolution to Agentic Architecture

The financial technology landscape is currently undergoing a radical transformation, shifting from static, rule-based automation tools—characterized as "Generation 2.0"—toward dynamic, reasoning-capable systems known as "Generation 3.0" Agentic AI.¹ The Smart Trade Agentic AI System is engineered specifically to embody this shift, functioning not merely as a passive execution script but as a cognitive orchestration layer that resides between the retail trader and the market infrastructure.

The architectural imperative of this system is to bridge the "Syntax Cliff," a documented cognitive gap where traders possess sophisticated strategic intent but lack the linguistic capability to translate that intent into the rigid syntactic requirements of proprietary trading platforms like MetaTrader 5 (MQL5) or TradingView (Pine Script).¹ Traditional solutions have attempted to solve this via "No-Code" drag-and-drop builders, but these introduce a "Complexity Ceiling," preventing the expression of stateful, nuanced logic.¹ Conversely, the Smart Trade architecture adopts an **Agentic** approach. In this paradigm, the software is composed of autonomous functional agents—Perception, Reasoning, Validation, and Execution—that collaborate to interpret natural language, deduce missing parameters through clarification loops, and rigorously validate logic against historical data before any market interaction occurs.²

This system is strictly bounded as a "Strategic Translation and Validation Engine." It creates a verifiable "Grey Box" environment where the internal probabilistic reasoning of Large Language Models (LLMs) is converted into deterministic, inspectable code. This ensures that the user retains intellectual ownership and trust, countering the "Black Box" anxiety prevalent in the current market.¹ The following analysis provides an exhaustive breakdown of this architecture, deconstructing the system into its atomic components, data structures, and inter-process communication protocols to demonstrate how it achieves high-fidelity strategy automation.

1.2 High-Level System Topology

The system is architected as a distributed, microservices-oriented application containerized within a Docker environment to ensure portability and reproducibility.¹ The topology consists

of three primary layers:

- 1. **The Presentation Layer (Perception):** A stateful web client responsible for capturing user intent and rendering complex financial data.
- 2. **The Application Layer (Cognition & Orchestration):** A Python-based backend serving as the central nervous system. This layer hosts the Agentic reasoning loops, the Retrieval-Augmented Generation (RAG) pipeline, the Backtesting Engine, and the Risk Sentinel.
- 3. **The Infrastructure Layer (Execution & Persistence):** This includes the database cluster (PostgreSQL/Redis/Pinecone) and the specialized Execution Bridge that maintains a persistent socket connection to the MetaTrader 5 terminal.¹

Layer	Primary Responsibility	Key Technologies
Presentation	Intent Capture, Visualization, Clarification	React, Redux, WebSockets
Application	Orchestration, Reasoning, Validation	FastAPI, Celery, LangChain, Backtrader
Data	Persistence, Vector Indexing, Caching	PostgreSQL, Pinecone, Redis
Execution	Signal Transmission, Market Feedback	TCP Sockets, MetaTrader5 Python Library

2. The Perception Layer: Frontend Engineering and Intent Capture

The Perception Layer is the system's sensory interface. Unlike standard trading dashboards that function as passive command centers, this layer is designed as an active "Perception Module".² It is responsible for ingesting raw, unstructured natural language data, sanitizing it, and managing the state of the conversational workflow.

2.1 The Secure Web Gateway (Client Architecture)

The client is built using a component-based architecture (React), structured to handle the asynchronous nature of AI-driven interactions. The state management strategy is critical here, as the client must synchronize the user's conversational state with the backend's reasoning state.

2.1.1 Conversational Input Interface (CII)

The CII is the atomic unit for user interaction. It is not a simple text input field but a complex state machine that transitions between Listening, Processing, Clarification, and Error states.

- **Sub-Component: Input Sanitization Module** Before any text is transmitted to the backend, it passes through a client-side sanitization filter. This module utilizes Regular Expressions (Regex) to strip potentially malicious prompt-injection patterns (e.g., "Ignore previous instructions," "System Override").¹ This serves as the first line of defense, ensuring that the downstream LLM receives only clean, intent-focused text.
- **Sub-Component: State Synchronizer** This component utilizes the WebSocket protocol to maintain a persistent connection with the backend orchestration layer. It listens for state_change events. For instance, when the backend identifies ambiguity in a strategy (e.g., "Timeframe undefined"), the State Synchronizer triggers a UI transition, replacing the input field with a specific "Clarification Card" component.¹

2.1.2 Visualization Engine (The "Glass Box")

To satisfy the "Grey Box" transparency requirement ¹, the frontend includes a dedicated Visualization Engine. This subsystem is responsible for parsing the complex JSON payloads returned by the backend and rendering them into human-readable formats.

- **Sub-Component: Code Differential Viewer**
When the RAG pipeline generates or refines MQL5/Pine Script code, the backend sends the raw string to the client. The Code Differential Viewer utilizes syntax highlighting libraries (e.g., PrismJS or Monaco Editor) to render the code. Crucially, it includes a "Diff" algorithm to visually highlight changes between iterations (e.g., changing a generic Moving Average to an Exponential Moving Average), allowing the user to visually verify the translation of their intent.
- **Sub-Component: Metric Renderer** This component receives the "Robustness Score" and backtesting metrics (Sharpe Ratio, Max Drawdown). It uses data visualization libraries (e.g., D3.js or Chart.js) to plot the Equity Curve and Drawdown Chart. It transforms raw statistical arrays into interactive graphs, enabling the user to zoom in on specific stress periods (e.g., the COVID-19 crash) identified during the validation phase.¹

2.2 Frontend-Backend Interface Contract

The communication between the Perception Layer and the Application Layer is governed by a strict API contract to ensure data integrity and type safety.

Endpoint	Method	Payload Structure (JSON)	Function
/api/v1/intent	POST	{ "session_id":	Transmits raw user

		"uuid", "text": "string", "context": }	intent for classification.
/api/v1/clarify	POST	{ "session_id": "uuid", "answer": "string", "variable_id": "string" }	Returns user response to specific ambiguity questions.
/api/v1/socket	WS	Stream<Event>	Bi-directional stream for status updates and alerts.

3. The Orchestration Core: Backend Application Architecture

The Application Layer is the "Cognitive Module" of the system.² It is architected using the FastAPI framework, chosen for its high-performance asynchronous capabilities (ASGI) and native support for Pydantic data validation.⁴ This layer acts as the orchestrator, delegating tasks to specialized subsystems (NLP, Validation, Execution) while managing the overall lifecycle of the user's strategy.

3.1 The Request Orchestrator Middleware

Upon receiving a request from the Perception Layer, traffic flows through a series of middleware components designed to secure and contextualize the request before it reaches the core logic.

3.1.1 Security and Isolation Subsystem

- **Authentication Guard (JWT):** This module intercepts every HTTP request. It decodes the JSON Web Token (JWT) in the header to extract the user_id. This ID is then injected into the request scope, ensuring that all subsequent database queries are scoped strictly to that user, satisfying the "Multi-User Isolation" requirement.¹
- **Rate Limiter:** To protect the expensive computational resources (LLM tokens and Backtesting CPU cycles), a sliding-window rate limiter is implemented using Redis. It tracks the request frequency per user_id and rejects bursts that exceed the defined quota (e.g., 5 strategy generations per minute).

3.1.2 Dependency Injection Container

The system utilizes FastAPI's dependency injection system to manage the lifecycle of database connections and external service clients.⁵

- **Database Session Manager:** Provides a transactional scope for PostgreSQL operations. If an error occurs during strategy generation, the session manager automatically rolls back the transaction to maintain data consistency.
- **Vector DB Client:** Manages the connection pool to the Pinecone vector database, ensuring that embedding queries are efficient and thread-safe.

3.2 The Asynchronous Task Manager (Celery & Redis)

Strategy generation and backtesting are long-running operations that cannot block the main web thread. The system employs a Producer-Consumer architecture using Celery as the task queue and Redis as the message broker.

3.2.1 Task Producer

When a user requests a new strategy, the API endpoint does not execute the logic directly. Instead, it serializes the request parameters and pushes a `generate_strategy_task` onto the Redis queue. It immediately returns a `task_id` to the frontend, allowing the UI to poll for status updates.⁶

3.2.2 Worker Pools

The system segregates workers based on their resource profiles:

- **GenAI Workers:** These workers handle I/O-bound tasks involving the LLM and Vector Database. They are configured with high concurrency limits since they spend most of their time waiting for network responses.
- **Quant Workers:** These workers handle CPU-bound tasks like Backtesting and Optimization. They are configured with a strict 1 process per core limit to prevent context switching overhead during heavy numerical calculations (NumPy/Pandas operations).¹

4. The Cognitive Engine: NLP and RAG Pipeline

This subsystem represents the "Reasoning Engine" of the agentic architecture.² It is responsible for the semantic processing of user intent, the retrieval of grounded knowledge, and the generation of syntactic code. It explicitly addresses the "Hallucination" risk inherent in LLMs by enforcing a strict Retrieval-Augmented Generation (RAG) workflow.¹

4.1 The Intent Classification and Routing Module

Before generation begins, the raw natural language input is analyzed to determine the appropriate workflow.

4.1.1 The Semantic Router

This component utilizes a lightweight classifier model (or a specific zero-shot prompt) to categorize the user's input into one of several intent buckets:

- **STRATEGY_CREATION:** The user is describing a new trading logic (e.g., "Create a MACD strategy").
- **STRATEGY_REFINEMENT:** The user is modifying an existing strategy (e.g., "Change the stop loss to 50 pips").
- **CLARIFICATION_RESPONSE:** The user is answering a system-generated question.
- **EXPLANATION_REQUEST:** The user wants to understand a specific part of the code.

4.1.2 The Ambiguity Detection Subsystem

A critical requirement of the system is the "Clarification Loop".¹ This module scans the input for missing or vague parameters that are essential for a deterministic strategy. It employs a checklist-based validation logic:

- **Logic Check:** Does the input define *Entry* conditions?
- **Logic Check:** Does the input define *Exit* conditions?
- **Risk Check:** Is a *Stop Loss* mechanism defined?
- **Context Check:** Is the *Timeframe* and *Asset Class* specified?

If any of these checks fail, the module triggers the Ambiguity state. It generates a specific clarification question (e.g., "You mentioned a 'Trend', how do you technically define that? SMA 200? EMA 50?") and halts the generation process until the user responds.

4.2 The Retrieval-Augmented Generation (RAG) Pipeline

This pipeline ensures that the AI generates code that is syntactically valid for the specific target platform (MQL5 or Pine Script).⁷

4.2.1 Vector Database Architecture (Pinecone)

The system maintains a specialized knowledge base stored as vector embeddings in Pinecone.¹

- **Data Ingestion & Chunking:** Authoritative documentation (e.g., the MQL5 Reference Manual, Pine Script v5 User Guide) and "Golden Template" code snippets are ingested. The text is split into semantic chunks (approximately 512 tokens) with a sliding window overlap (e.g., 50 tokens) to preserve context across boundaries.⁸
- **Embedding Model:** The chunks are converted into high-dimensional vectors using an embedding model optimized for code (e.g., text-embedding-ada-002 or a specialized HuggingFace code model).
- **Namespace Isolation:** The index is partitioned into namespaces (mql5_docs, pine_docs, risk_templates) to ensure that a query for MQL5 syntax does not retrieve Pine Script

results.⁹

4.2.2 The Context Assembly Module

When a generation task is triggered, this module acts as the "Prompt Engineer."

1. **Query Generation:** It converts the user's intent into a search query vector.
2. **Retrieval:** It queries Pinecone for the Top-K (e.g., 5) most relevant code snippets and documentation chunks.
3. **Prompt Construction:** It assembles a dynamic prompt that includes:
 - The User's specific intent.
 - The Retrieved "Golden Templates" (e.g., correct syntax for `iRSI()` or `strategy.entry()`).
 - System Instructions (e.g., "Enforce strict typing," "Include comments").
 - Mandatory Risk Logic injection blocks.

4.3 The Code Generation and Self-Correction Loop

This is the final stage of the cognitive process, where the structured prompt is converted into executable code.

4.3.1 The Deterministic Code Generator

To minimize risk, the system does not allow the LLM to write the *entire* file from scratch. Instead, it uses a "Scaffolding" approach.

- **Skeleton Injection:** The system provides a pre-validated code skeleton containing the necessary import statements, class structures (for MQL5), and event loop definitions (`OnTick()`).
- **Logic Injection:** The LLM is tasked with generating only the specific *logic blocks* (e.g., the if conditions for entry/exit) which are then injected into the safe skeleton. This ensures that the core structural integrity of the script is maintained.¹

4.3.2 The "Compiler Loop" (Self-Correction)

Recognizing that LLMs can still produce syntax errors, the system implements a recursive self-correction mechanism.¹

1. **Draft Generation:** The LLM produces `Draft_Code_v1`.
 2. **Static Analysis:** The draft is passed to a specialized parser (see Section 5.1).
 3. **Feedback:** If the parser detects errors (e.g., "Undeclared identifier"), the error log is fed back into the LLM as a new prompt: "The code you generated has the following error: [Error]. Please fix it."
 4. **Iteration:** This loop repeats up to 3 times. If the code still fails validation, the task is marked as FAILED, and the user is prompted to simplify the strategy.
-

5. The Validation Subsystem: Adversarial Backtesting

Once a strategy is generated, it must pass through the Validation Subsystem before it is eligible for deployment. This layer functions as an "Adversarial Agent," actively trying to break the strategy to ensure it is robust enough for live markets.¹

5.1 The Static Analysis Validator (Safety Gate)

This module performs a static code analysis *without* running the strategy, acting as a gatekeeper against logical fallacies and safety violations.

5.1.1 Look-Ahead Bias Detector

One of the most common failures in algorithmic trading is "Look-Ahead Bias," where the strategy inadvertently uses data from the future (e.g., the Close price of the *current* bar) to make a decision.¹

- **Mechanism:** The validator parses the code Abstract Syntax Tree (AST). It scans for array indexing patterns that reference index 0 (current bar) in a context that implies a finalized value. In Pine Script, it checks for the usage of `security()` calls with `lookahead=barmerge.lookahead_on`, which causes repainting. If detected, the strategy is rejected with a specific error message explaining the bias.

5.1.2 Parameter Validity Checker

This component enforces the "Risk Guardrails" at the code level.¹

- **Stop Loss Check:** It verifies that a Stop Loss mechanism is present. It checks the calculated distance of the SL; if it is smaller than the broker's minimum `StopsLevel` (a defined constant in the system), it flags the strategy as "Too Sensitive."
- **Position Sizing Cap:** It scans the logic for volume calculation. It asserts that the logic cannot produce a volume that exceeds the system-wide hard cap (e.g., 2% of equity).

5.2 The Backtesting Engine Architecture

The core simulation is powered by a high-performance backtesting library (Backtrader or vectorbt) integrated into the Python worker environment.¹⁰

5.2.1 Data Ingestion and Synchronization Unit

Garbage in, garbage out. This unit is responsible for providing clean data to the simulator.

- **Data Handler:** It fetches historical OHLCV data from the database (TimescaleDB).
- **Integrity Check:** Before passing data to the engine, it scans for "Data Gaps" (missing timestamps) and "Bad Ticks" (price = 0 or infinity).
- **Synchronization:** It ensures that the historical data used for the test matches the data feed of the target execution environment (MT5) to prevent the "Backtest-Reality Gap".¹

5.2.2 The Simulation Core

The engine runs the strategy logic through the historical data, candle by candle.

- **Broker Simulator:** This sub-component models the friction of the real world.¹⁰
 - *Slippage Model:* It applies a probabilistic penalty to execution prices, simulating the delay between signal generation and order fill.
 - *Spread Model:* It uses variable spread data (if available) or a conservative average spread to calculate entry/exit prices.
 - *Commission Model:* It deducts transaction costs from every trade to calculate Net Profit.

5.3 The Adversarial Stress-Test Module

This module goes beyond standard backtesting by subjecting the strategy to "Torture Tests".¹

5.3.1 Stress Scenarios

The system maintains a library of adverse market conditions:

- **Volatility Shock:** The simulation creates a synthetic dataset where the ATR (Average True Range) is multiplied by 2x or 3x. This tests if the Stop Loss logic can survive extreme noise.
- **Liquidity Crisis:** The simulation widens the spread by 5x during specific windows. This tests if the strategy remains profitable when transaction costs spike.
- **Black Swan Replay:** The strategy is run specifically through historical crash periods (e.g., the 2020 COVID crash, the 2008 Financial Crisis) to assess its "Tail Risk."

5.3.2 Robustness Scoring Engine

Finally, the results are aggregated into a composite "Robustness Score" (0-100).¹ This is not just a measure of profit, but of safety.

- **Math:** $Score = (w_1 \cdot Sharpe) + (w_2 \cdot (1 - MaxDD)) + (w_3 \cdot Stability)$
- **Thresholds:**
 - If Max Drawdown > 50%, the Score is set to 0 (Automatic Failure).
 - If Win Rate < 30%, the Score is penalized heavily (unless Risk/Reward is very high).
 - If the strategy creates < 10 trades over a year, it is flagged for "insufficient sample size."

6. The Execution Bridge: Connectivity and Protocols

The Execution Layer is the physical interface to the market. Unlike passive charting tools, this system must actively manage orders. To avoid the fragility of third-party webhooks (latency,

downtime), the system employs a custom-built, persistent **Execution Bridge** connecting the Python backend to the MetaTrader 5 terminal.¹³

6.1 The Socket Communication Architecture

The bridge utilizes a TCP/IP socket connection for Inter-Process Communication (IPC). This design minimizes latency and provides immediate feedback, unlike HTTP polling.¹⁴

6.1.1 The Socket Server (Python Side)

Running within the Docker container, the Python backend initiates a socket server listening on a specific port (e.g., 5555).

- **Protocol:** JSON-over-Socket. All messages are serialized as JSON strings encoded in UTF-8.
- **Connection Handling:** It supports a single persistent connection per strategy instance to ensure order of execution.

6.1.2 The Socket Client (MT5 Side)

An Expert Advisor (EA) running inside the MT5 terminal acts as the client. It connects to the Python server on startup.

- **Event Loop:** The EA runs inside the OnTick() or OnTimer() event. It continuously listens for incoming JSON payloads from the Python server.
- **Command Parsing:** When a payload arrives, the EA deserializes the JSON and routes the command to the appropriate MQL5 function (e.g., OrderSend).

6.2 The MetaTrader 5 Connector Library

For direct interaction with the terminal from the Python environment (when running on the same Windows host or compatible container), the system utilizes the official MetaTrader5 Python library.¹⁵

6.2.1 Atomic Functions

- **Initialization:** `mt5.initialize()` establishes the link to the specific terminal instance.
- **Authentication:** `mt5.login(login, password, server)` authenticates the trading account.
- **Data Fetching:** `mt5.copy_rates_from_pos(symbol, timeframe, start_pos, count)` is used to pull historical data for the backtesting engine.
- **Order Execution:** `mt5.order_send(request)` is the critical function. The system wraps this function in a try-except block to handle return codes (e.g., 10009 for "Request Completed", 10004 for "Requote").

6.3 Fail-Safe and Heartbeat Mechanisms

Reliability is paramount. The system implements a Heartbeat Protocol to detect connection

failures immediately.¹

6.3.1 The Heartbeat Protocol

- **Ping/Pong:** Every 5 seconds, the Python server sends a {"type": "PING"} message.
- **Response:** The MT5 EA must respond with {"type": "PONG", "status": "OK"} within a 2-second window.
- **Failure State:** If the response is missed (timeout), the Python system increments a failure counter. If 3 consecutive heartbeats are missed, the system declares **Bridge Failure**.

6.3.2 The Safe-State Protocol

Upon declaring a Bridge Failure, the system triggers the "Safe State" logic ¹:

1. **Alerting:** The user is immediately notified via the Web Client (WebSocket push) and Email.
2. **Signal Block:** The Strategy Engine is locked; no new signals are generated.
3. **Reconnection Logic:** The system attempts to restart the socket connection loop in a separate thread while keeping the strategy halted.

7. The Sentinel: Risk Management Middleware

The "Sentinel" is a specialized middleware component dedicated entirely to risk enforcement. It sits between the Strategy Engine and the Execution Bridge, acting as a final filter for all outgoing orders. Its mandate is to enforce the "Hard Boundaries" of safety, overriding the strategy logic if necessary.¹

7.1 Pre-Trade Risk Validation

Before any OrderSend payload is transmitted to the bridge, the Sentinel intercepts it and performs synchronous validation checks.

Risk Check	Logic Description	Action on Failure
Max Position Size	Checks if $\text{Volume} * \text{ContractSize} * \text{Price} > \text{AccountEquity} * \text{MaxRisk\%}$ (e.g., 2%).	Scale Down: Reduces volume to the max allowed limit.
Mandatory Stop Loss	Verifies sl field is not null and $\text{sl} > 0$.	Inject: Calculates and inserts a default SL (e.g., 2x ATR).

Fat Finger Filter	Checks if Volume > System_Max_Lot_Limit (e.g., 10.0 lots).	Reject: Blocks the order completely.
Frequency Limit	Checks time since last trade. Prevents runaway algo loops (HFT prevention).	Reject: Blocks orders sent < 1s apart.

7.2 The Emergency Kill Switch (Kernel Level)

The Kill Switch is the ultimate safety mechanism.¹⁹ It is implemented as a high-priority interrupt signal.

- **Trigger Mechanism:** It can be triggered manually by the user (Dashboard Button) or automatically by the Sentinel (e.g., if Daily_Drawdown > 5%).
- **Execution Sequence:**
 1. **Queue Freeze:** The Sentinel immediately locks the outgoing command queue.
 2. **Cancel All:** It constructs a {"action": "CANCEL_ALL_PENDING"} payload and sends it with High_Priority flag.
 3. **Close All:** It iterates through the list of open positions (fetched via mt5.positions_get()) and generates market close orders for each one.
 4. **Process Termination:** Once confirmed, it terminates the Strategy Execution Thread to prevent any logic from restarting.

8. Data Persistence and Infrastructure Layer

The system's memory and state are managed by a robust infrastructure layer designed for data integrity and auditability.

8.1 Database Schema (PostgreSQL)

The relational database stores the configuration, strategy definitions, and audit logs. The schema is designed to support the "Requirements Traceability Matrix" (RTM).¹

- **Table: strategies**
 - id (UUID): Primary Key.
 - user_id (FK): Owner.
 - intent_raw (Text): The original natural language prompt.
 - spec_json (JSONB): The structured strategy specification.
 - code_mql5 (Text): The generated source code.
 - robustness_score (Float): The validation result.

- status (Enum): DRAFT, VALIDATED, ACTIVE, HALTED.
- **Table: audit_logs**
 - id (BigInt): Primary Key.
 - timestamp (ISO8601): Precision time of event.
 - event_type (Enum): SIGNAL_GEN, ORDER_SENT, RISK_REJECT, KILL_SWITCH.
 - payload (JSONB): The exact data packet associated with the event.
 - strategy_id (FK): Link to the strategy.

8.2 Time-Series Data (TimescaleDB)

For efficient storage of market data (ticks and candles), the system utilizes TimescaleDB, an extension of PostgreSQL optimized for time-series data.

- **Hypertable: market_data**
 - Partitioned by time and symbol to ensure fast queries for backtesting.
 - Used to cache historical data fetched from MT5 to avoid redundant API calls.

8.3 Containerization and Deployment

The entire architecture is defined as a multi-container application using Docker Compose, ensuring isolation and ease of deployment.

- **Network:** A private bridge network `smart_trade_net` connects the containers.
- **Volumes:** Persistent volumes are mapped for the Database (`pg_data`) and Redis (`redis_data`) to ensure data survives container restarts.
- **Services:**
 - `backend_api`: The FastAPI orchestration core.
 - `celery_worker`: The compute nodes for GenAI and Backtesting.
 - `db`: PostgreSQL/TimescaleDB.
 - `redis`: Message broker.
 - `frontend`: Nginx serving the React build.

9. System Integration: The Complete Lifecycle

To visualize how these components fit together, we trace the complete lifecycle of a strategy from interface to execution.

1. **Perception (Interface):** The user logs into the Web Client and types: "Create a strategy that buys EURUSD when the 50 SMA crosses above the 200 SMA."
2. **Cognition (NLP & RAG):**
 - The **Request Orchestrator** routes this to the **NLP Engine**.
 - The **Ambiguity Detector** notes a missing "Stop Loss."
 - The **Clarification Loop** triggers, asking the user: "What is your risk rule?"
 - User replies: "Use a 50 pip trailing stop."

- The **Context Assembler** retrieves MQL5 syntax for Moving Average and Trailing Stop from **Pinecone**.
- The **LLM** generates the MQL5 code.
- The **Static Analysis Validator** confirms no syntax errors.
- 3. **Validation (Simulation):**
 - The **Backtesting Engine** pulls EURUSD history from **TimescaleDB**.
 - The **Simulation Core** runs the strategy, including a **Volatility Shock** stress test.
 - The **Robustness Engine** assigns a score of 85/100.
- 4. **Deployment (Execution):**
 - User clicks "Deploy."
 - The **Strategy Engine** spins up a thread, subscribing to EURUSD ticks via the **Execution Bridge**.
 - A tick arrives. Logic triggers a BUY signal.
 - The **Sentinel** verifies the trade size (0.1 lots) is within the 2% risk cap.
 - The **Socket Server** transmits {"action": "ORDER_SEND",...}.
 - The **MT5 Client** executes the trade and returns {"ticket": 12345}.
- 5. **Persistence (Audit):**
 - The **Audit Log** records the entire chain: User Prompt -> Generated Code -> Validation Score -> Live Trade Ticket.

This integrated flow demonstrates the system's ability to seamlessly translate abstract human intent into concrete, validated, and safe market actions, fulfilling the core architectural mandate of the Smart Trade Agentic AI System.

Works cited

1. Academic Requirements Specification.pdf
2. Agentic AI Architecture: Types, Components & Best Practices - Exabeam, accessed February 11, 2026, <https://www.exabeam.com/explainers/agentic-ai/agentic-ai-architecture-types-components-best-practices/>
3. Tech Navigator: Agentic AI Architecture and Blueprints - Infosys, accessed February 11, 2026, <https://www.infosys.com/iki/research/agentic-ai-architecture-blueprints.html>
4. Ultimate guide to FastAPI library in Python - Deepnote, accessed February 11, 2026, <https://deepnote.com/blog/ultimate-guide-to-fastapi-library-in-python>
5. Bigger Applications - Multiple Files - FastAPI, accessed February 11, 2026, <https://fastapi.tiangolo.com/tutorial/bigger-applications/>
6. Structuring a FastAPI Project: Best Practices - DEV Community, accessed February 11, 2026, <https://dev.to/mohammad222pr/structuring-a-fastapi-project-best-practices-5316>
7. RAG Pipelines Explained: Setup, Tools, and Strategies | Generative AI Collaboration Platform, accessed February 11, 2026,

- <https://orq.ai/blog/rag-pipelines>
8. RAG Pipeline: Example, Tools & How to Build It - lakeFS, accessed February 11, 2026, <https://lakefs.io/blog/what-is-rag-pipeline/>
 9. Understanding the RAG Pipeline: Components and Hyperparameters | by Ajay Verma, accessed February 11, 2026, <https://medium.com/@ajayverma23/understanding-the-rag-pipeline-components-and-hyperparameters-66772b7ede56>
 10. How I Built an Event-Driven Backtesting Engine in Python | by Timothy Kimutai | Medium, accessed February 11, 2026, <https://timkimutai.medium.com/how-i-built-an-event-driven-backtesting-engine-in-python-25179a80cde0>
 11. Build a custom backtester with Python - AlgoTrading101 Blog, accessed February 11, 2026, <https://algotrading101.com/learn/build-my-own-custom-backtester-python/>
 12. The Fed - Proposed 2026 Stress Test Scenarios - Federal Reserve Board, accessed February 11, 2026, <https://www.federalreserve.gov/publications/2025-november-proposed-2026-stress-test-scenarios.htm>
 13. Price Action Analysis Toolkit Development (Part 36): Unlocking Direct Python Access to MetaTrader 5 Market Streams - MQL5 Articles, accessed February 11, 2026, <https://www.mql5.com/en/articles/19065>
 14. Integrating Python and ML Algorithms with MT5 for Forex Trading: Worth it? - Reddit, accessed February 11, 2026, https://www.reddit.com/r/algorithmictrading/comments/1cqxdju/integrating_python_and_ml_algorithms_with_mt5_for/
 15. Python Integration - MQL5 Reference, accessed February 11, 2026, https://www.mql5.com/en/docs/python_metatrader5
 16. metatrader5 - PyPI, accessed February 11, 2026, <https://pypi.org/project/metatrader5/>
 17. Algorithmic trading: enhancing your systems, governance and controls - KPMG International, accessed February 11, 2026, <https://assets.kpmg.com/content/dam/kpmgsites/uk/pdf/2019/12/algorithmic-trading.pdf>
 18. Risk Management Systems in Algorithmic Trading: A Comprehensive Framework - NURP, accessed February 11, 2026, <https://nurp.com/wisdom/risk-management-systems-in-algorithmic-trading-a-comprehensive-framework/>
 19. Best Practices For Automated Trading Risk Controls And System Safeguards - FIA.org, accessed February 11, 2026, https://www.fia.org/sites/default/files/2024-07/FIA_WP_AUTOMATED%20TRADING%20RISK%20CONTROLS_FINAL_0.pdf